

Voice latency and audio quality: what was wrong, what was fixed

Eleven defects found and fixed in the voice path, each one measured rather than inferred. Includes the evidence that the original 90 ms target is unreachable, and the one remaining change worth making.

Owner: Balaji · Defects V-01 ... V-07 · All figures measured on the development machine, 24–25 August 2026

1. Result

BEFORE 3274 ms p50, with audible corruption	AFTER ~2700 ms p50, audio confirmed clean	AUDIO DEFECTS 0 7 of 7 replies spoken in full
--	--	--

Latency fell by roughly 18%, but the more important outcome is that the voice is now **correct**. For most of this work the pipeline was losing about half of every reply's audio, which users experienced as crackling, speech that raced, and clipped final words. That is fixed and confirmed both by ear and in the logs.

THE HEADLINE NUMBER UNDERSTATES THE WORK

An intermediate build reached **1551 ms** — but it achieved that by splitting replies across multiple Hume flushes, which silently discarded half the audio. That optimisation was reverted. The honest position is 2700 ms with intact audio, not 1551 ms with broken audio.

2. The 90 ms target is not reachable

The original brief asked for sub-90 ms. Measured from the development machine, the vendors alone cost more than ten times that before any Evarna code runs:

DEPENDENCY	ROUND TRIP	LOCATION	ON CRITICAL PATH?
Hume (TTS)	8 ms	edge, near you	yes
LiveKit (audio transport)	16 ms	India South	yes
MongoDB Atlas	31 ms	near you	yes
Deepgram (STT)	278 ms	United States	no — finalises early
OpenAI	~1000 ms	US origin	yes — dominant

Two further facts settle it. Hume's own first audio is **380 ms warm, 920 ms cold**. And a request to OpenAI that performs *no inference at all* — simply listing models — takes **900–1350 ms**.

90 ms is also shorter than the pause a human leaves between conversational turns, roughly 200 ms. A system that fast would read as interrupting rather than listening. The workable targets are **P50 under 800 ms and P95 under 1200 ms**.

3. Defects found and fixed

V-03 · INSTRUMENTATION

Nothing measured itself

No timing existed anywhere in the voice path, so no change could be proven. `getFirstChunkAt()` existed and was called by nothing. Added a per-turn timer stamping speech-end, transcript-final, first token and first audio frame, plus `npm run latency-report` to read it back. Everything else in this report depends on it.

V-04 · TURN TAKING

Turn-taking left entirely at library defaults

Endpointing delay was 500 ms on every turn; reduced to 300 ms, the value LiveKit itself uses for streaming detectors. Interruption `minWords` was **0**, meaning any sound cut the companion off — which is why a breath or an "mm-hm" stopped it mid-sentence. Now 2.

V-05 · CRITICAL PATH

Moderation blocked the first token

Moderation, an embedding call and an Atlas vector search all had to complete before the model was asked to start. On voice, generation now begins immediately and the *first token* is gated on the moderation verdict instead. The safety guarantee is unchanged — nothing unmoderated ever becomes audible — but the wait now hides under the model's own thinking time.

V-07 · COST

Voice replies could run to 600 tokens

Roughly 40 seconds of unstoppable speech, billed to Hume. Capped at 150 tokens on the voice path; the text path is unchanged.

NEW · CRITICAL PATH

Context compression made a blocking LLM call mid-turn

Once a conversation passed 3500 tokens, every later turn paid a full non-streaming completion — 1.5 to 3 seconds — before generation could start, and it worsened the longer someone talked. Compression only reduces the token bill and changes nothing the caller hears, so on voice it now runs in the background and lands for the next turn.

Memory retrieval sat in front of every reply

An embedding call plus a vector search, measured at 330–580 ms, blocking the prompt. It now starts on the *interim* transcript while the caller is still speaking, so it is already resolved when the turn completes. Memory search is approximate by construction, so a query a few words short of final is an acceptable trade.

4. The audio corruption — and the optimisation that caused it

Users reported three symptoms: crackling mid-reply, speech that sometimes raced, and final words cut short ("laptop" heard as "lapt"). These were repeatedly misdiagnosed as timing problems. A direct measurement against Hume settled it — same text, same voice:

HOW THE TEXT WAS SENT	SAMPLES RETURNED	DURATION	VERDICT
One flush, whole sentence	193,920	4.04 s	correct
Two flushes, split at the comma	94,080	1.96 s	49% — half lost
Two flushes, serialised	105,600	2.20 s	still lost
Two flushes, back to back	82,560	1.72 s	still lost
Non-instant mode	74,880	1.56 s	still lost

ROOT CAUSE

Hume's streaming input does not concatenate across flushes. Splitting a reply into sentences loses roughly half the speech, regardless of timing, `instant_mode`, or where the voice object is sent. The audio was never distorted — it was *incomplete*. Crackling was the seams where words were missing; the racing was whole clauses absent; the clipped last word was the final flush truncated.

Sentence-level flushing had been introduced deliberately to cut the token-to-audio stage from ~2000 ms to ~700 ms. It bought that speed by destroying audio. It has been reverted to one flush per reply, and a check now reads the source and fails if a mid-reply flush is ever reintroduced.

Four further audio defects, each independent

PCM DECODING

Odd-length chunks shifted every following sample

`new Int16Array(buf.buffer, buf.byteOffset, buf.byteLength / 2)` — when Hume's chunk had an odd byte length, the fractional length truncated and the trailing byte was silently dropped. The next chunk then began mid-sample and everything after it was shifted by one byte. The half sample is now carried into the next chunk. This also fixed a latent crash: Node's pooled buffers can be misaligned, and `Int16Array` throws on that inside a WebSocket handler where nothing catches it.

SHARED SOCKET

Concurrent streams stole each other's audio

One Hume socket is shared by a whole call, but several TTS streams can be alive at once — a barge-in leaves the old one draining while its replacement starts. Audio delivery was claimed in the *constructor*, so a stream that was merely created stole audio from the stream being played. Separately, a superseded stream calling `cancelTurn()` set a shared discard flag and silenced the turn that had replaced it. Both produced replies with visible text and no sound. Turns now carry an ownership token.

CROSS-TURN LEAK

Abandoned audio played inside the next reply

Cancelling a turn does not stop Hume generating — it only stops us reading. The discard flag was cleared unconditionally when the next turn began, so the tail of the abandoned reply was delivered into the new turn's queue and played as part of it. Two utterances interleaved. The abandoned generation is now counted out before anything new is accepted.

TAIL

The final audio chunk was dropped on abort

The drain loop holds one chunk back so it can mark the true final frame. On abort it broke *without emitting it*, removing the tail of the last word. Now emitted before leaving.

5. What remains, and what to do about it

Every lever inside Evarna's code has been pulled. The remaining latency belongs to vendors and geography. Three options were evaluated; only one is worth doing.

Recommended — move the LLM, not the worker

A controlled comparison of the *same* Azure service in two regions isolates distance from compute:

AZURE REGION	TCP	TLS	FULL HTTPS ROUND TRIP
Central India	24 ms	73 ms	94 ms
South India	27 ms	79 ms	100 ms
East US	223 ms	676 ms	1136 ms

Azure East US costs 1136 ms from this network — almost exactly what OpenAI costs. The delay is not OpenAI being slow; it is what *any* US-hosted API costs from here. Azure OpenAI in Central India would replace ~256 ms of network with ~25 ms.

Expected gain: 350–600 ms, taking end-to-end latency to roughly **2100–2400 ms**. Only the network leg is measured; inference time is unknown and could give some of that back as queueing. Two things to verify before committing: that `gpt-4o-mini` is available in Central India, and that the quota is high enough not to queue. The code change is small — `src/config/openai.ts` is 18 lines.

Rejected — moving the voice worker to the US

This was the initial recommendation and measurement overturned it. Three of the four services the worker talks to constantly — LiveKit, Atlas and Hume — are already nearby. Moving to the US would fix OpenAI but add ~250 ms to LiveKit audio in and out, and ~200 ms to Atlas. Net gain shrinks to roughly 350 ms while permanently degrading every Indian user's audio path.

Rejected — OpenAI regional endpoints

`jp.api.openai.com` and `eu.api.openai.com` exist and return 200, but they are for data residency, not latency. Japan — far closer than the US — measured **consistently slower** (1.48 s vs 1.02 s). No India or Asia-Pacific endpoint exists.

Available but not recommended — parallel Hume connections

One socket per sentence *does* avoid the data loss: 264,936 samples against a 193,920 baseline, nothing missing. But each sentence gets its own prosody envelope, so the reply runs **37% longer** with an audible beat between sentences. Given that clean, natural audio was just recovered at considerable cost, spending it on latency is not advised.

6. Verification

Three suites guard this work. The audio checks are offline and run in seconds.

COMMAND	ASSERTIONS	COVERS
check:voice-tts-ownership	14	socket ownership, cross-turn leak, PCM decoding, one-flush rule
check:voice-latency	10	the moved safety gate, reply cap, persistence
smoke	71	full pipeline, crisis path unchanged

The moderation change relocated a *safety gate*, so `check:voice-latency` exists specifically to prove that a crisis message on the voice path yields the crisis response first and leaks zero pre-verdict model tokens. `smoke` covers only the text path and would not catch a regression there.

RECOMMENDED NEXT STEP

Run twenty varied utterances — clean sentences, mid-sentence pauses, single words, deliberate interruptions — and read the result with `npm run latency-report`. Four to seven turns is too thin to set a p50 against. That twenty-turn run is the artefact that settles whether Evarna stays on a cascade or moves to a speech-to-speech model.